

# FSOC Virtual Camera Tracking Simulator

Coarse Pointing, Acquisition and Tracking (PAT) of an optical beacon  
with learned beacon identification

Technical Report · software version 0.2.1 · 2 October 2026

**Abstract.** Free-space optical communication (FSOC) links need their two terminals to point at each other to within a fraction of a milliradian. Before a fine-steering mirror can take over, a *coarse* stage must find the partner terminal's beacon in a wide field of view, confirm that it is the right light, and keep it near boresight on a gimbal with limited slew rate. This report describes a complete software simulation of that coarse stage. A virtual pan-tilt camera renders a synthetic sky with a blinking beacon, decoy lights, stars, clouds, atmospheric turbulence, platform vibration and sensor noise. A closed loop of blob detection, beacon identification, Kalman tracking and PID/feed-forward gimbal control acquires and tracks the beacon, and every frame is logged against ground truth. Identification combines a classical blink-frequency test with *BeaconNet*, a 62,818-parameter convolutional network that classifies a 0.8 s video clip of each candidate; on held-out scenes that include harmonic decoys it reaches ROC-AUC 0.972 versus 0.882 for the blink test alone. In a 40-seed evaluation on random scenes with 0–5 decoys each, the hybrid identifier acquires the true beacon in 92% of runs (brightest-dot baseline: 62%) and reduces runs with sustained false locks from 42% to 8%, at a processing cost of about 8 ms per frame. The report documents every model and algorithm as implemented, the evaluation method and results, and the known limitations.

## Contents

|                                   |    |
|-----------------------------------|----|
| 1. Introduction                   | 2  |
| 2. System Architecture            | 2  |
| 3. Virtual World and Camera Model | 4  |
| 4. Disturbance Models             | 5  |
| 5. Candidate Detection            | 6  |
| 6. Beacon Identification          | 6  |
| 7. Training the Network           | 8  |
| 8. Tracking                       | 9  |
| 9. Pointing Control and Search    | 10 |
| 10. Performance Monitoring        | 11 |
| 11. Results                       | 12 |
| 12. User Interfaces and Tooling   | 14 |
| 13. Limitations and Future Work   | 15 |
| 14. Conclusion                    | 16 |

## 1. Introduction

An FSOC terminal transmits data on a narrow laser beam, typically a few hundred microradians wide or less. Link establishment follows the **PAT** sequence: *pointing* the terminal open-loop at the partner's reported position (from GPS or ephemeris data), *acquisition* of the partner's beacon by searching the uncertainty region, and closed-loop *tracking*. Tracking is normally split into a coarse loop (camera plus gimbal, wide field of view, low bandwidth) and a fine loop (quadrant detector plus fast-steering mirror, narrow field, high bandwidth). The coarse loop's job is to bring the beacon inside the fine loop's capture range and keep it there.

The coarse loop faces problems that are hard to reproduce in the field and hard to test repeatably: atmospheric scintillation that makes the beacon twinkle and fade, angle-of-arrival jitter, platform vibration, clouds that hide the beacon for seconds, sun glints and glare, sensor noise, hot pixels, and other lights of similar brightness (aircraft, street lamps, other terminals) that the tracker must *not* lock onto. This project provides a software test bench for all of them.

### 1.1 Objectives

- Simulate a virtual pan-tilt camera looking at a moving optical beacon, with physically motivated disturbances that can be switched on, off and scaled individually.
- Implement a complete coarse PAT pipeline: search, detection, identification, Kalman tracking, coasting through occlusions, re-acquisition and gimbal control.
- Recognise the designated beacon among look-alike lights from its temporal signature (a 4 Hz blink code), using both a classical signal-processing test and a trained neural network.
- Measure performance against ground truth (acquisition time, lock retention, pointing error, false locks, processing time) and report it automatically for every run, with reproducible seeds.
- Offer a desktop application for interactive demonstration and a headless mode for batch evaluation.

### 1.2 Coarse versus fine pointing

This project implements the **coarse** stage only, as the problem statement requires. Table 1 shows why every design parameter of the simulator falls in the coarse regime; the fine stage is represented by the handover criterion (Section 10).

*Table 1. Coarse and fine pointing stages of an FSOC terminal.*

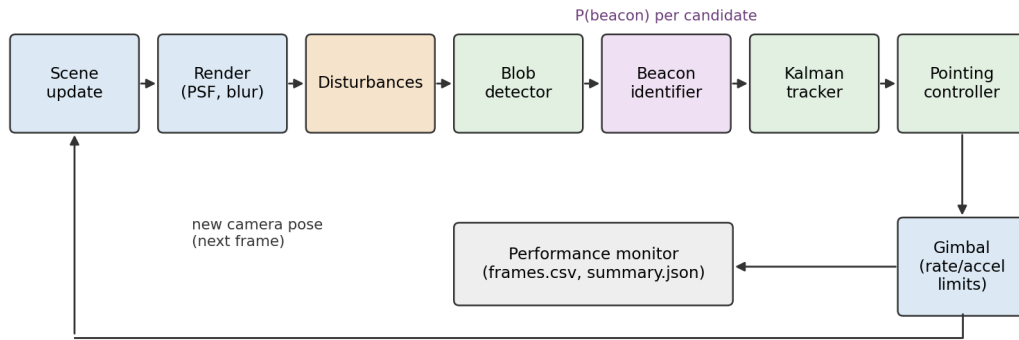
| Property        | Coarse stage (this project)                                                                                    | Fine stage (not simulated)                            |
|-----------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Actuator        | Pan-tilt gimbal, $\leq 25^\circ/\text{s}$                                                                      | Fast-steering mirror / piezo                          |
| Sensor          | Camera, $640 \times 480$ , $8^\circ \times 6^\circ$ field of view                                              | Quadrant detector or PSD, $\approx 0.1^\circ$ or less |
| Field of regard | $\pm 30^\circ \times \pm 20^\circ$ (wide sky: $\pm 60^\circ \times \pm 30^\circ$ ) with spiral / raster search | A few mrad                                            |
| Accuracy        | $\approx 0.2\text{--}2$ mrad (1 px = 0.218 mrad)                                                               | $\approx 1\text{--}10$ $\mu\text{rad}$                |
| Loop rate       | 30 Hz                                                                                                          | 1–10 kHz                                              |
| Task            | Find, identify and centre the beacon; hand over within 10 px (2.2 mrad)                                        | Remove residual jitter so the laser hits the receiver |

### 1.3 Scope of this report

Sections 2–9 describe the implementation module by module, with the equations and parameter values that the code actually uses. Section 10 defines the metrics, Section 11 presents measured results, Section 12 the user interfaces and tooling, and Section 13 the limitations. All angles are in degrees unless stated otherwise; with the default camera one pixel corresponds to  $0.0125^\circ = 0.218$  mrad.

## 2. System Architecture

The code base (about 5,800 lines of Python, of which roughly 2,300 form the simulation core in the `fsoc` package) is organised as small single-purpose modules wired together by one GUI-independent class, `Simulation`. One call to `Simulation.step()` produces one camera frame and advances the closed loop by one time step (Figure 1).



**Figure 1.** Per-frame closed loop implemented in `fsoc/simulation.py`. The gimbal command computed from frame  $k$  moves the camera during the interval before frame  $k+1$ .

**Table 2.** Modules of the simulator.

| Module                              | Responsibility                                                                                                                                                                                    |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>config.py</code>              | Loads <code>config/default.yaml</code> and deep-merges a scenario file over it (dicts merge key by key; lists such as the target list are replaced).                                              |
| <code>targets.py</code>             | Beacon and decoys; trajectories: static, linear, Lissajous, circular, random walk; on/off blink modulation.                                                                                       |
| <code>scene.py</code>               | Renders only the visible part of the sky: stars, glints and targets as Gaussian PSFs with per-source motion blur; <code>render_frame</code> composes a frame in physical depth order (Section 4). |
| <code>camera.py</code>              | Pan-tilt camera: field of view, small-angle projection, slew-rate and acceleration limits, travel stops.                                                                                          |
| <code>disturbances.py</code>        | Vibration, turbulence (scintillation, wander, blur, shimmer), clouds, dropouts, glints, glare, sensor noise.                                                                                      |
| <code>detector.py</code>            | Adaptive threshold, region-of-interest threshold, connected components, sub-pixel centroid.                                                                                                       |
| <code>identification.py</code>      | Candidate tracklets, blink SNR test, CNN scorer (ONNX via OpenCV DNN), evidence accumulation.                                                                                                     |
| <code>tracker.py</code>             | Constant-velocity Kalman filter, gating, SEARCH / ACQUIRING / TRACKING / COASTING state machine, blacklist.                                                                                       |
| <code>controller.py</code>          | PID with velocity feed-forward while tracking; predictive spiral and raster search otherwise.                                                                                                     |
| <code>metrics.py</code>             | Ground-truth metrics per frame, run summary, <code>frames.csv</code> / <code>summary.json</code> / <code>summary.txt</code> .                                                                     |
| <code>randomizer.py</code>          | Random scene generation from the run seed.                                                                                                                                                        |
| <code>visualization.py, gui/</code> | OpenCV overlay; PySide6 desktop application with 3D view, 2D sky map, live plots and a disturbance explainer.                                                                                     |
| <code>tools/</code>                 | Dataset generation, CNN training and ONNX export, method comparison, multi-seed evaluation.                                                                                                       |

## 2.1 Timing and causality

The simulation runs at a fixed rate of 30 frames/s ( $\Delta t = 33.3$  ms). Within a step the world advances first, then the frame is rendered from the camera's *actual* line of sight (commanded pose plus vibration offset). Detections are converted to world angles with the *commanded* pose, because a real tracker only knows its encoder readings; vibration therefore appears to the tracker as apparent target motion, as it would in hardware. The controller output is applied to the gimbal after the frame, which introduces the one-frame latency of a real camera-in-the-loop system.

## 2.2 Configuration and reproducibility

Every parameter lives in `config/default.yaml`, grouped by module, and every disturbance has an enabled switch and a strength multiplier. Six scenario files list only the values that differ from the defaults. Each run uses a single integer seed: with `seed: null` a fresh seed is drawn and written to the report, so any run can be replayed exactly with `--seed`. The scene randomiser uses its own generator derived from the same seed, so the scene and the noise are both reproducible.

### 3. Virtual World and Camera Model

#### 3.1 Coordinates and projection

The world is a patch of sky of  $\pm 30^\circ$  in azimuth and  $\pm 20^\circ$  in elevation. The camera has a  $640 \times 480$  sensor and an  $8^\circ \times 6^\circ$  field of view, giving 80 pixels per degree on both axes. Because the field of view is small, the projection is linear (small-angle): for a camera pointing at  $(az_c, el_c)$

$$x = c_x + (az - az_c) \cdot ppd_x, \quad y = c_y - (el - el_c) \cdot ppd_y$$

with  $(c_x, c_y)$  the image centre and  $ppd$  the pixels per degree. The inverse mapping converts detections back to world angles. Working in world coordinates is central to the design: target motion and camera motion are separated, so the tracker's velocity estimate is the target's angular velocity and not an artefact of panning.

**Angles versus distances.** Like a real camera, the simulator works purely in angles; a degree has the same meaning as on a real terminal. The linear size that an angle covers depends on the range to the target:  $1^\circ$  (80 px) is 17 m at 1 km, 175 m at 10 km and 8.7 km at 500 km (low Earth orbit), so one pixel is 0.22 m, 2.2 m and 110 m respectively. The  $\pm 30^\circ \times \pm 20^\circ$  patch is not meant to be the whole sky: a real terminal points at the partner's GPS / ephemeris position and only searches the uncertainty region around it, which is a few degrees. The simulated beacon speeds of 0.8–5  $^\circ/s$  correspond to 14–87 m/s for a drone at 1 km, 140–870 m/s for an aircraft at 10 km, and include the 0.5–1  $^\circ/s$  of a low-orbit satellite passing overhead.

#### 3.2 Gimbal dynamics

The camera is driven by angular-rate commands. Each axis first limits the change in rate to  $a_{\max} \cdot \Delta t$  (80  $^\circ/s^2$ ) and then clips the rate to  $\pm 25^\circ/s$ , then integrates the position. Hard stops at the world boundaries zero the rate. The controller therefore cannot teleport the camera; the search pattern and the tracking gains have to respect these limits.

#### 3.3 Rendering

Instead of cropping a large pre-rendered image, `Scene.render` draws only the sources that fall inside the current view. Each source is a 2-D Gaussian point-spread function at a sub-pixel position ( $\sigma = 2$  px for targets, 0.7 px for stars), evaluated only in a  $\pm 3\sigma$  window. Six hundred background stars with a power-law brightness distribution (peak 10–60 grey levels, below the detection floor) provide realistic clutter that moves across the image when the camera pans.

**Motion blur** is computed per source from its motion *relative to the camera* during the exposure (10 ms). The source is drawn as a Gaussian 'line' along the streak vector  $u = (v_{src} - v_{cam}) \cdot ppd \cdot t_{exp}$ . To conserve flux the peak is reduced to

$$peak_{eff} = peak / (1 + L / (\sqrt{(2\pi) \sigma}))$$

where  $L$  is the streak length in pixels. A target the camera follows therefore stays sharp while stars streak, exactly as in a real tracking camera, and the beacon smears and dims during fast search slews. Turbulence blur widens  $\sigma$  to  $\sqrt{(\sigma^2 + \sigma_b^2)}$  and scales the peak by  $(\sigma/\sigma')^2$ , again conserving flux.

#### 3.4 Targets and trajectories

Every light source is a `Target` with an intensity, a spot size, an optional blink (frequency, duty cycle, off level, phase) and a trajectory. Exactly one target must be the beacon. The default beacon blinks at **4 Hz with a 70% duty cycle and dims to 30% when 'off'**, so it never disappears completely yet carries a clear temporal code. Five trajectory types are available (Table 3).

**Table 3.** Trajectory models.

| Type        | Model                                                                                                                                                                                                                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| static      | Fixed position.                                                                                                                                                                                                                                                                      |
| linear      | Constant velocity, reflected at the world edges.                                                                                                                                                                                                                                     |
| lissajous   | az, el = c + A·sin(2πft + φ) per axis; smooth figure-of-eight motion.                                                                                                                                                                                                                |
| circular    | Orbit of radius r and period T about a centre (e.g. a loitering UAV).                                                                                                                                                                                                                |
| random_walk | Constant speed; the <i>turn rate</i> is an Ornstein–Uhlenbeck process (σ, τ), so the target banks into and out of turns. Near the edge it commits to a turn away from the nearest wall at up to 3 rad/s; the edge margin is at least 2.2 turning radii so that a U-turn always fits. |

### 3.5 Random scenes

With `randomize.enabled` (the default for interactive use) each run draws a beacon path speed (0.8–5 °/s) for a random-walk path (smooth random manoeuvres; Lissajous and circular paths remain available), a beacon intensity, 2–5 decoys (at least two, so that identification is always exercised; half of them blinking at deliberately wrong rates: 0.5–2.5 Hz or 6–10 Hz), turbulence strength 0.5–2.0, vibration strength 0.3–2.0, clouds with probability 0.5, and an **initial pointing error** of 3–10° in a random direction. The last item models the open-loop pointing step of PAT: the partner's *reported* position (GPS / ephemeris) is wrong by a few degrees. The report is a live stream (it moves with the partner, offset by its error). The camera always starts at the centre of the sky, slews to the reported position and spirals around it; after a long loss the search returns to it instead of scanning the whole sky. **Depth:** every light and cloud has a relative distance (0 = camera, 1 = stars) and a cloud dims or hides only the lights *behind* it; the beacon's distance drifts during a run (a bounded random walk), so it passes in front of and behind clouds. The sky can be enlarged to az ±60° × el ±30°; path sizes then stay those of the standard sky (a target does not fly a bigger pattern because the camera can look further), paths may lie anywhere in the larger sky, and the numbers of stars and clouds grow with the area so that their density is unchanged.

## 4. Disturbance Models

All stochastic disturbances use first-order Gauss–Markov (discrete Ornstein–Uhlenbeck) processes rather than independent per-frame noise, because real turbulence and jitter are temporally correlated:

$$x_{k+1} = a \cdot x_k + \sqrt{(1 - a^2) \cdot \sigma} \cdot n_k, \quad a = \exp(-\Delta t / \tau), \quad n_k \sim N(0, 1)$$

This keeps the stationary standard deviation equal to σ for any time step, with correlation time τ.

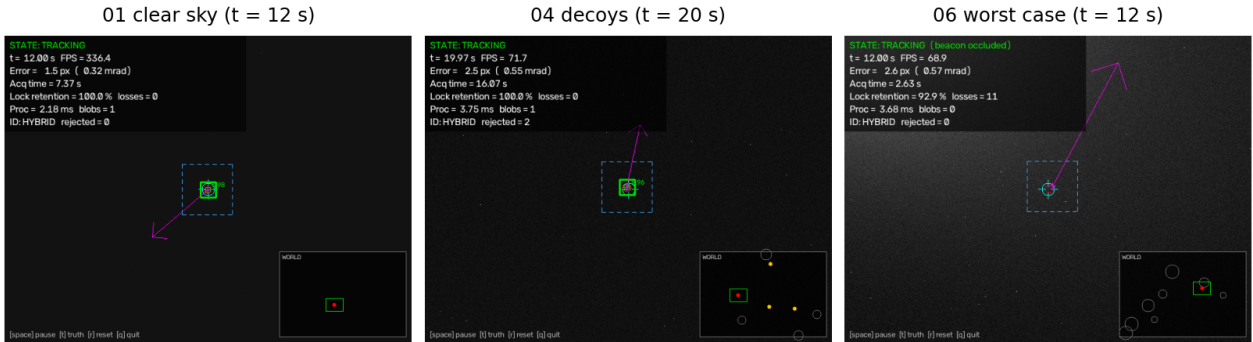
**Table 4.** Disturbance models. Each has an enabled switch and a strength multiplier.

| Effect             | Physical cause                  | Implementation (default parameters)                                                                                                                                                                                           |
|--------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform vibration | Engine, rotor, structural modes | Two sinusoidal tones (7 Hz, 0.010°; 23 Hz, 0.005°) plus broadband OU jitter (0.006°, τ = 30 ms) added to the line of sight. Its rate also feeds motion blur.                                                                  |
| Scintillation      | Refractive-index turbulence     | Log-normal intensity gain exp(χ − σ²/2) with unit mean; χ is OU with σ = 0.3, τ = 30 ms, independent per source.                                                                                                              |
| Beam wander        | Angle-of-arrival jitter         | Per-source OU position offset, 0.8 px, τ = 0.1 s.                                                                                                                                                                             |
| Turbulence blur    | Beam spreading                  | Extra PSF width 0.6 px added in quadrature.                                                                                                                                                                                   |
| Heat shimmer       | Local image distortion          | OU displacement field on a 16 × 12 grid (0.6 px, τ = 0.15 s), cubic upsampling, applied with <code>cv2.remap</code> .                                                                                                         |
| Clouds             | Drifting cloud banks            | Gaussian optical-depth blobs moving with the wind and wrapping around; transmission T = exp(−τ); sky and stars: image = (img − b) · T + b with cloud glow b = 35; each light is dimmed only by the clouds nearer than itself. |
| Dropouts           | Obstruction, beacon power dips  | Poisson events (mean interval 15 s) that blank the beacon for 0.5–2 s.                                                                                                                                                        |
| Glints             | Sun reflections, flares         | Poisson flashes (0.3 Hz) inside the FOV lasting 0.05–0.4 s, peak 150–255.                                                                                                                                                     |
| Glare              | Sun or moon near the FOV        | Wide Gaussian background (level 120, σ = 3°) fixed in the world.                                                                                                                                                              |
| Sensor             | Camera electronics              | Shot noise σ = 0.5√signal plus read noise 3 grey levels, 25 hot pixels, Poisson single-pixel hits; 8-bit saturation.                                                                                                          |

Order matters and follows the physics. `render_frame` composes each frame from far to near: first the sky background, the stars and the glints (which belong to the far scene); then the clouds, which dim and veil everything

drawn so far; then the beacon and decoys, each with its per-source effects (scintillation, wander, blur, dropouts) and dimmed only by the clouds nearer than itself; finally the whole-image effects: glare, heat shimmer and sensor noise. A light in front of a cloud therefore stays bright while one behind it fades or vanishes. For speed, the sensor model draws one bank of unit Gaussian noise twice the frame size at start-up and takes a random crop each frame; visually this is indistinguishable from fresh noise and removes the most expensive per-frame operation. Turbulence, vibration and sensor strength can be changed while a run is in progress (the GUI sliders); a disturbance that was off is created on demand.

A frame counts as **occluded** when a dropout is active or cloud transmission at the beacon is below 50%, at which point the dimmed beacon falls below the detection threshold. Occluded frames are reported separately so that the tracker is not blamed for what no camera can see.



**Figure 2.** Frames produced by the simulator with the OpenCV overlay (truth markers on). Left: clear sky, locked on the beacon. Centre: decoy scenario, locked on the beacon (green box with its  $P(\text{beacon})$  score) while three decoys are visible on the world minimap (yellow). Right: worst-case scenario, seed 42, tracking the beacon through a cloud ('beacon occluded'); clouds (grey circles on the minimap) and the raised noise floor are visible. The magenta arrow is the Kalman velocity estimate (0.5 s ahead).

## 5. Candidate Detection

The detector (BlobDetector) finds every bright spot in the 8-bit frame and leaves the decision of *which* spot is the beacon to later stages.

- **Smoothing:** a  $3 \times 3$  Gaussian blur suppresses single-pixel noise and hot-pixel hits.
- **Adaptive threshold:** background  $b$  = median and robust noise  $\sigma_n = 1.4826 \cdot \text{MAD}$  are estimated on a  $4 \times$  sub-sampled image; threshold =  $\max(b + 8\sigma_n, 80)$ . The median and MAD are barely affected by the few bright spots, and sub-sampling makes the estimate cheap.
- **Region of interest:** inside an  $80 \times 80$  px window around the tracker's predicted position a much lower threshold is used,  $\max(b + 5\sigma_n, 45)$ . The beacon's blink-off phase (30% power) and deep scintillation fades are still detected there, while the rest of the image is not flooded with stars and noise.
- **Connected components** (8-connectivity) with an area filter of 2–400 px.
- **Sub-pixel centroid** weighted by brightness above background on the *un-blurred* frame, plus peak and integrated flux per blob.

Detections are then converted to world angles with the commanded camera pose.

## 6. Beacon Identification

In a single frame the beacon, the decoys and bright stars are indistinguishable Gaussian spots. What differs is their behaviour over time: the beacon blinks at a known rate (its 'code'), decoys are steady or blink at other rates, glints vanish within a fraction of a second, and hot pixels stay fixed on the sensor while the sky moves. Identification therefore works on short histories of each candidate.

## 6.1 Candidate tracklets

CandidateTracker follows *every* detection from frame to frame in world coordinates with greedy nearest-neighbour association (closest pairs first). Each tracklet predicts with a smoothed velocity; its association gate starts at  $0.25^\circ$  and widens by 50% per missed frame (capped at  $1^\circ$ ) because a blinking or fading source becomes less certain while unseen. Every frame, *including frames in which the candidate was not detected*, a  $24 \times 24$  px patch is cut at the predicted position with sub-pixel interpolation (`cv2.getRectSubPix`), and the aperture flux ( $7 \times 7$  box minus the patch median) is stored. Recording the off-frames is what makes the blink visible. A tracklet is ‘ready’ once it holds 24 patches (0.8 s); the flux history is kept twice as long (1.6 s) for sharper frequency analysis.

## 6.2 Classical blink test

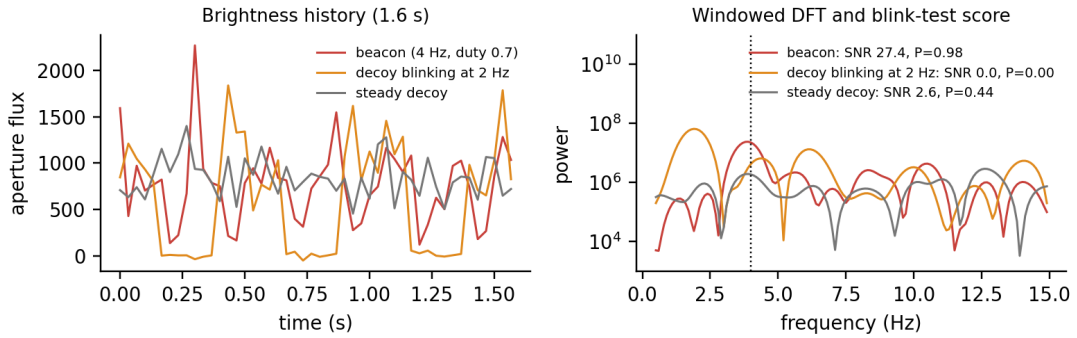
BlinkScorer evaluates a Hann-windowed single-bin DFT of the flux history at the beacon frequency  $f_0 = 4$  Hz and on a 0.25 Hz grid of ‘noise’ frequencies that excludes  $\pm 1$  Hz around  $f_0$  and  $2f_0$ . The test statistic is a spectral signal-to-noise ratio

$$\text{SNR} = P(f_0) / \text{mean } P(f_{\text{noise}}) \cdot \min(1, P(f_0) / \max_{k \in \{2,3,4\}} P(f_0/k))$$

Broadband flicker such as scintillation raises signal and noise bins equally and cancels out. The second factor handles **harmonic decoys**: a light blinking as a square wave at  $f_0/2$  has a harmonic exactly at  $f_0$ ; such a light has more power at its fundamental than at  $f_0$  and is penalised. The SNR is mapped to a probability with a logistic calibrated on validation data, where the beacon's median SNR was about 7 and every decoy type's median 0.5–1.5:

$$p = 1 / (1 + \exp(-4.2 \cdot (\log_{10} \text{SNR} - \log_{10} 3)))$$

so SNR 1  $\rightarrow$  0.1, SNR 3  $\rightarrow$  0.5 and SNR 10  $\rightarrow$  0.9. The DFT matrices are cached per window length, so scoring all candidates is one small matrix product.



**Figure 3.** Blink test on synthetic 1.6 s flux histories with scintillation ( $\sigma = 0.3$ ) and additive noise, scored with the project's BlinkScorer. The beacon's 4 Hz peak gives a high SNR; the 2 Hz decoy is rejected by the harmonic check; a single clip of a steady light can reach an intermediate score, which is why single-clip scores are accumulated over time (Section 6.5).

## 6.3 BeaconNet: learned identification

CNNScorer runs a small convolutional network that treats the candidate's last  $T = 24$  patches as a 24-channel image, i.e. a short video clip. The first convolution therefore learns spatio-temporal filters that combine the blink pattern, the spot shape and the consistency of the motion in one step. Before inference each clip is normalised by subtracting its median and dividing by its maximum (at least 20), then clipped to  $[-0.5, 1.5]$ ; this keeps the *relative* temporal pattern while making the network insensitive to absolute beacon power or cloud dimming.

**Table 5.** BeaconNet architecture (`tools/train_classifier.py`).

| Layer                                                                   | Output shape             | Parameters    |
|-------------------------------------------------------------------------|--------------------------|---------------|
| Input clip ( $T = 24$ frames)                                           | $24 \times 24 \times 24$ | —             |
| Conv $3 \times 3$ ( $24 \rightarrow 32$ ) + BatchNorm + ReLU, MaxPool 2 | $32 \times 12 \times 12$ | 7,008         |
| Conv $3 \times 3$ ( $32 \rightarrow 64$ ) + BatchNorm + ReLU, MaxPool 2 | $64 \times 6 \times 6$   | 18,624        |
| Conv $3 \times 3$ ( $64 \rightarrow 64$ ) + BatchNorm + ReLU            | $64 \times 6 \times 6$   | 37,056        |
| Global average pool, Dropout 0.3, Linear ( $64 \rightarrow 2$ )         | 2 logits                 | 130           |
| <b>Total</b>                                                            |                          | <b>62,818</b> |



At run time the model is loaded from `models/beacon_cnn.onnx` with OpenCV's DNN module, so PyTorch is not needed to run the application. All ready tracklets are scored in one batch; a softmax over the two logits gives  $P(\text{beacon})$ . If the model file is missing, the simulation prints a warning and falls back to the blink test.

## 6.4 Methods

Four identification methods are selectable: **hybrid** (default: the mean of CNN and blink probabilities), **cnn**, **blink**, and **none** (no identification: the tracker takes the brightest candidate, the baseline). The hybrid is the default because on validation scenes it made the fewest false 'this is the beacon' calls: the CNN is better at recognising the beacon, the blink test better at rejecting wrong-rate decoys (Table 8), and averaging lets each veto the other's mistakes.

## 6.5 Evidence accumulation

A single 0.8 s clip can be misleading (a deep fade, a blink hidden in noise). Each clip probability, clamped to [0.02, 0.98], is converted to log-odds  $\ell$  and averaged into a per-tracklet evidence value with a leaky running mean

$$e_n = e_{n-1} + \alpha(\ell_n - e_{n-1}), \quad \alpha = \max(1/n, 1/90), \quad \text{score} = 1/(1 + e^{-e_n})$$

This is an exact mean for the first 90 clips and an exponential memory of about 3 s afterwards: consistent evidence builds up quickly, single outliers barely move the verdict, and a verdict can still change if the tracker has been following the wrong light. A detection reports its score only after at least 24 scored clips (1.6 s of flux history, enough to tell 2 Hz from 4 Hz); before that its score is 0.5 ('undecided').

# 7. Training the Network

## 7.1 Dataset generation

Labels come for free because the simulator knows where every source is. `tools/generate_dataset.py` runs randomised episodes (15 s each) using the *same* rendering, detection and candidate-tracking code as the application. Each episode randomises the beacon's waveform (duty 0.6–0.8, off level 15–45%), intensity, spot size and trajectory; adds 2–6 decoys drawn from four types (steady; clearly different rate 0.4–2.5 or 5.5–12 Hz; near rate 2.5–3.0 or 5.0–5.5 Hz; harmonic at  $f_0/2$ ,  $/3$  or  $/4$ ); and randomises every disturbance (turbulence up to  $3\times$ , vibration up to  $2.5\times$ , glints up to 3 Hz, clouds, glare and dropouts with some probability). A virtual operator moves the camera around a focus point that switches between the beacon and random decoys, with a random wander range and speed, so both steady tracking and fast slewing are represented.

Every fourth frame, each ready tracklet becomes a sample: its 24-patch clip, the label (beacon or not) and the kind of source it sat on. A tracklet is matched to the nearest true source within  $0.12^\circ$ ; samples whose window is less than 80% consistent are discarded as ambiguous. Train and validation sets are split **by episode**, so validation scenes are never seen in training. Episodes run in parallel worker processes.

**Table 6.** Generated datasets (sample counts by source kind). 'Other' = stars, hot pixels and noise.

| Dataset                    | Samples | Validation | Beacon | Steady | Wrong rate | Near rate | Glint | Harmonic | Other |
|----------------------------|---------|------------|--------|--------|------------|-----------|-------|----------|-------|
| v1 (beacon_dataset.npz)    | 17,953  | 4,281      | 4,506  | 2,636  | 2,044      | 735       | 0     | 0        | 8,032 |
| v2 (beacon_dataset_v2.npz) | 16,218  | 2,467      | 4,764  | 1,760  | 1,912      | 733       | 4     | 759      | 6,286 |

## 7.2 Training procedure

Training uses PyTorch with class-weighted cross-entropy (to balance the minority beacon class), AdamW (learning rate  $2 \cdot 10^{-3}$ , weight decay  $10^{-4}$ ) with a one-cycle schedule, batch size 128 and 25 epochs. Augmentation applies random horizontal and vertical flips,  $90^\circ$  rotations (the problem is rotation invariant) and small additive noise. The checkpoint with the lowest validation loss is kept, exported to ONNX (opset 13, dynamic batch) and checked against OpenCV's DNN: the maximum logit difference is  $1.2\text{e-}06$ , i.e. the deployed model reproduces PyTorch exactly. By default a retrain writes to `models/candidate/` so that it never silently replaces the model the application uses.



### 7.3 Validation results

**Table 7.** Per-clip classification on 2,467 held-out validation clips of dataset v2 (shipped model v2, models/training\_report.json). Single clips, before evidence accumulation.

| Metric                        | BeaconNet (CNN)       | Classical blink test   |
|-------------------------------|-----------------------|------------------------|
| Accuracy                      | 0.907                 | 0.853                  |
| Precision                     | 0.802                 | 0.781                  |
| Recall                        | 0.929                 | 0.726                  |
| F1 score                      | 0.861                 | 0.752                  |
| ROC-AUC                       | 0.972                 | 0.882                  |
| Confusion (TP / FP / TN / FN) | 708 / 175 / 1530 / 54 | 553 / 155 / 1550 / 209 |

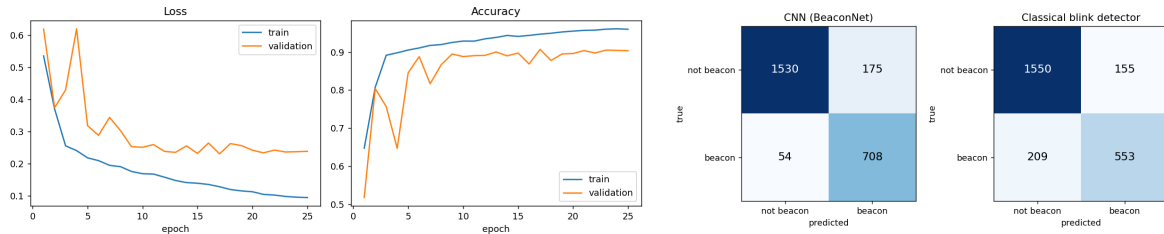
**Table 8.** Accuracy per candidate type on the validation set. The two methods fail on different kinds of candidate, which motivates the hybrid.

| Candidate type   | n   | CNN accuracy | Blink accuracy |
|------------------|-----|--------------|----------------|
| beacon           | 762 | 0.929        | 0.726          |
| steady decoy     | 293 | 0.891        | 0.898          |
| wrong rate decoy | 333 | 0.790        | 0.934          |
| near rate decoy  | 272 | 0.919        | 0.801          |
| harmonic decoy   | 269 | 0.818        | 0.926          |
| other            | 538 | 0.996        | 0.946          |

The CNN is clearly better at recognising the beacon (recall 0.93 vs 0.73) and at rejecting stars, hot pixels and noise (0.996 vs 0.946); the blink test is better on decoys blinking at a clearly wrong rate (0.934 vs 0.790) and on harmonic decoys (0.926 vs 0.818), thanks to its sub-harmonic check. The CNN costs about 0.32 ms per candidate on a CPU. These are single-clip figures; in the closed loop the accumulated score over up to 3 s is far more reliable.

### 7.4 Model versions

The first model (v1) was trained on dataset v1, which contains no harmonic decoys. After dataset v2 was generated, BeaconNet was retrained on it and compared with v1 on the *same* v2 validation set and in closed loop before being promoted. On the v2 validation clips the hybrid identifier's accuracy rose from 89.6% (v1) to 91.6% (v2), mainly on harmonic decoys; in a 40-seed closed-loop comparison both models locked for about 72% of the time (72.0% vs 71.5%) with fewer runs containing false locks (2 vs 3). v2 is therefore the model shipped in models/beacon\_cnn.onnx; v1 is kept in models/v1/. Its training report (ROC-AUC 0.976) was measured on the easier v1 validation set and is not directly comparable with Table 7. The multi-seed results in Section 11 were measured with v1, before the promotion; since v2 performed at least as well in closed loop on the same seeds, they remain representative.



**Figure 4.** Left: training and validation loss and accuracy per epoch. Right: validation confusion matrices of BeaconNet and of the classical blink detector.

## 8. Tracking

### 8.1 Kalman filter

The beacon is tracked in world angles with a 2-D constant-velocity Kalman filter, state  $x = [a_z, e_l, v_{a_z}, v_{e_l}]^T$ . The prediction uses  $F$  with  $F_{02} = F_{13} = \Delta t$  and the discrete white-noise acceleration model per axis

$$Q_{\text{axis}} = q^2 \cdot g g^T, \quad g = [\Delta t^2/2, \Delta t]^T$$

with  $q = 3 \text{ }^\circ/\text{s}^2$  by default (10–15  $^\circ/\text{s}^2$  in the fast-target scenarios). The measurement is the detection's position with  $R = (0.02^\circ)^2 \cdot \mathbf{I}_2$ , where  $\mathbf{I}_2$  is the  $2 \times 2$  identity matrix. The initial velocity variance is large (25  $(^\circ/\text{s})^2$ ) so the filter learns the target's velocity within a few frames.

## 8.2 Data association and gating

A detection is accepted when it is statistically consistent with the prediction, Mahalanobis distance  $d^2 = \mathbf{y}^T \mathbf{S}^{-1} \mathbf{y} < 11.6$  (99.7% for two degrees of freedom), or when it lies within a minimum gate of  $0.15^\circ$  of the prediction. The minimum gate keeps lock through sudden manoeuvres that the constant-velocity model did not expect; it grows by 50% per consecutive miss up to 4x. With identification enabled, a detection that belongs to the tracklet already being followed wins over any other candidate inside the gate. Detections that are blacklisted or 'clearly not the beacon' are removed before association.

## 8.3 Acquisition state machine

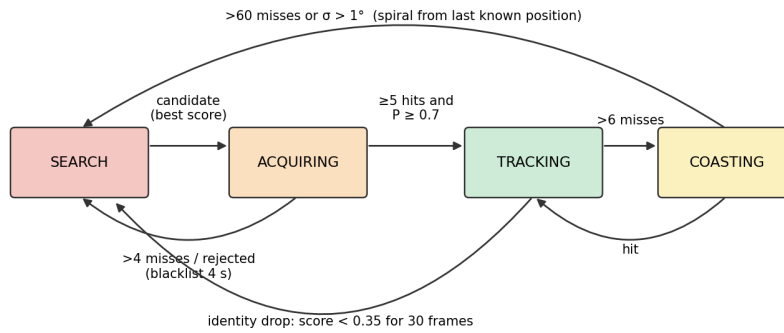


Figure 5. Tracker state machine (fsoc/tracker.py) with the default thresholds.

- **SEARCH:** no track. When candidates exist, the most beacon-like one (highest score, flux breaks ties) starts a tentative track.
- **ACQUIRING (verify):** the candidate is followed and verified. It is confirmed after at least 5 hits *and*, with identification, once its accumulated score is ready and  $\geq 0.7$ . It is abandoned after more than 4 consecutive misses, and *rejected and blacklisted for 4 s* if its score falls below 0.2 with at least 24 clips of evidence, or if it is not confirmed within 90 frames.
- **TRACKING:** confirmed lock. Up to 6 missed frames (blink-off, scintillation fades) are held without changing state. A lock whose score stays below 0.35 for 30 frames is dropped as a false lock and blacklisted.
- **COASTING:** the beacon is not seen (e.g. behind a cloud); the filter predicts only and the gimbal follows the prediction. A hit returns to TRACKING. Coasting ends after 60 frames or as soon as the position uncertainty exceeds  $1^\circ$ , whichever comes first; fast targets reach the uncertainty limit much sooner. The last position and velocity are handed to the search.

Rejecting a candidate requires more evidence than accepting it, because wrongly blacklisting the true beacon costs 4 s of blindness, while taking slightly longer to confirm costs little. The counters `rejections` and `dropped_locks` are included in every report.

## 9. Pointing Control and Search

### 9.1 Tracking control law

While a target estimate exists the controller commands the gimbal rate

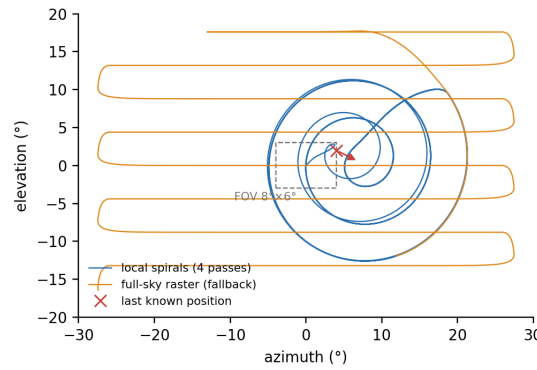
$$\omega = k_{ff} \cdot \hat{\mathbf{v}} + K_p \mathbf{e} + K_i \int \mathbf{e} \, dt + K_d \dot{\mathbf{e}}$$

where  $\mathbf{e}$  is the angle between the Kalman estimate and the boresight,  $\hat{\mathbf{v}}$  the estimated target velocity,  $k_{ff} = 1$ ,  $K_p = 5 \text{ s}^{-1}$ ,  $K_i = 0.5 \text{ s}^{-2}$ ,  $K_d = 0.05$  and the integral is clamped to  $\pm 2 \text{ }^\circ/\text{s}$  (anti-windup). Velocity feed-forward carries most of the load: the camera moves with the target, and the feedback terms only remove the residual offset. No extra one-frame lead is added to  $\mathbf{e}$  because the feed-forward term already accounts for motion over the next frame. The integral and derivative states are reset when the search starts.

## 9.2 Search strategy

In SEARCH the controller flies an Archimedean spiral  $r = s\theta/2\pi$  around its best guess of the beacon's direction. Before the first sighting this is the partner's reported position: in random scenes a live GPS / ephemeris stream that moves with the partner but is offset by its 3–10° reporting error. The camera starts at the centre of the sky, first slews straight to the reported position, and then spirals around it, the spiral centre following the report as it moves. After a loss the spiral is centred on the last known beacon position instead; fixed scenarios, which have no position report, always use the last known position or the initial pointing direction. The arm spacing  $s$  is  $0.8 \times$  the smaller FOV dimension (4.8°), so consecutive arms overlap and no sky is skipped;  $\theta$  is advanced so that the scan point moves at about 12 °/s, and the camera follows it with a proportional gain of 6 s<sup>-1</sup>. Three refinements make the search practical:

- **Predictive centre:** the spiral centre drifts along the beacon's last known velocity with a fading horizon,  $c(t) = c_0 + v \cdot \tau(1 - e^{-t/\tau})$ ,  $\tau = 3$  s, because a lost target keeps moving.
- **Bounded local search:** the spiral restarts beyond a 15° radius. After 4 local passes without success the search falls back to the live position report and spirals around it, which is much faster than scanning the whole sky; only if that also fails (or no report exists) is one full-sky raster ('lawnmower') pass flown, so that every direction is eventually covered, after which the local spiral resumes.
- **No reset on false alarms:** during ACQUIRING the spiral state is kept, so a glint that is briefly followed and then abandoned lets the scan resume where it left off instead of restarting.



**Figure 6.** Search path flown by the real controller and gimbal model for 90 s after a loss at (4°, 2°) with last velocity (1, -0.5) °/s and no position report (as in the fixed scenarios): four drifting local spirals (blue), then the full-sky raster fallback (orange). The dashed box shows one field of view.

## 10. Performance Monitoring

PerformanceMonitor compares the tracker with ground truth every frame and writes a per-frame log and a summary for every run. The definitions are deliberately strict: following a decoy is never counted as lock.

**Table 9.** Performance metrics (fsoc/metrics.py).

| Metric           | Definition                                                                                                              |
|------------------|-------------------------------------------------------------------------------------------------------------------------|
| Pointing error   | Angle between the true beacon and the camera's actual line of sight (including vibration), in degrees, mrad and pixels. |
| Locked           | Tracker in TRACKING and beacon in the FOV and estimate within 8 px of the true beacon.                                  |
| False lock       | Tracker in TRACKING but the estimate is not on the beacon.                                                              |
| Acquisition time | Simulated time until the first locked frame.                                                                            |
| Time locked      | Locked frames / all frames; also penalises slow or failed acquisition.                                                  |
| Lock retention   | Locked frames / frames after first lock; also reported excluding occluded frames.                                       |
| Handover ready   | Share of locked frames with pointing error ≤ 10 px (≈ 2.2 mrad): close enough for a fine-pointing stage.                |
| Processing time  | Wall-clock time of detection + identification + tracking + control per frame (rendering timed separately).              |

Each run folder contains frames.csv (21 columns per frame: state, lock flags, true, commanded, actual and estimated positions, errors, detection count, timings), summary.json and a human-readable summary.txt. The summary also records the seed, the random-scene description, the identification method and the numbers of rejected candidates and identity-dropped locks.

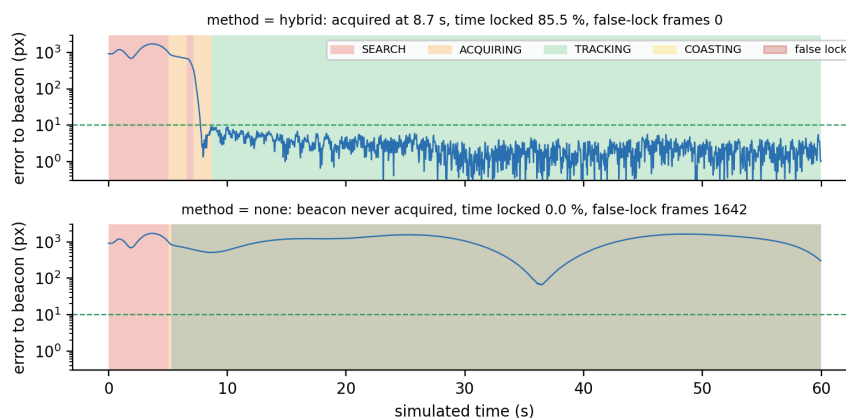
## 11. Results

### Key results at a glance

- **Acquisition:** hybrid AI identification finds the true beacon in **92%** of 40 random scenes, against 62% for the brightest-dot baseline.
- **False locks:** runs with a sustained lock on a decoy fall from **42% to 8%**.
- **Time on target:** 71% of each run locked on the beacon on average (baseline 54%), with a median error of 3.6 px (0.78 mrad) while locked, inside the 2.2 mrad handover zone.
- **Real time:** 8.1 ms of processing per frame, a quarter of the 33 ms frame budget.
- **Evidence:** 140 closed-loop runs of 60 s, every one reproducible from its seed.

### 11.1 A closed-loop run

Figure 7 shows the decoy scenario (one beacon, two steady decoys and one decoy blinking at 1 Hz, all of similar brightness) with seed 1, run once with the hybrid identifier and once with the brightest-dot baseline. With identification, the tracker rejects 1 candidate during verification, confirms the beacon at 8.7 s and holds it for the rest of the run with 0 lock losses: 85.5% of the run locked, mean error 2.6 px (0.57 mrad), maximum 8.9 px, and 100% of locked frames inside the handover zone. Without identification the tracker locks onto a steady decoy after the first search and follows it for the entire run (1642 false-lock frames, 0% locked on the beacon), which is exactly the failure the identification stage exists to prevent.



**Figure 7.** Error between the true beacon and the line of sight (log scale) with the tracker state as background, decoy scenario, seed 1, 60 s. Top: hybrid identification. Bottom: no identification. The dashed line marks the 10 px handover radius.

### 11.2 Multi-seed evaluation on random scenes

Because a single run can be lucky or unlucky, `tools/evaluate.py` repeats each configuration over many seeds; for random scenes every seed is a different beacon path, set of 0–5 decoys, atmosphere, cloud cover and initial pointing error. Each run lasts 60 s. Table 10 summarises 40 random scenes per method, with and without identification; both methods see exactly the same scenes (same seeds). The evaluation was run with CNN model v1 and a decoy range of 0–5; the application now draws at least two decoys per random scene (Section 3.5), the harder case in which identification matters most. Random scenes have since changed (camera starting at the sky centre, the beacon always flying a random walk, and depth for lights and clouds); a 20-seed check with these settings acquired the beacon in 18 of 20 standard and 19 of 20 wide-sky scenes.

**Table 10.** Random-scene evaluation, 40 seeds per method, 60 s per run (runs/evaluation/results.csv).

| Metric                                   | Hybrid identification | None (brightest dot) |
|------------------------------------------|-----------------------|----------------------|
| Runs                                     | 40                    | 40                   |
| Beacon acquired [95% Wilson CI]          | 92% [80–97]           | 62% [47–76]          |
| Acquisition time, median [IQR]           | 5.2 s [2.2–16.8]      | 2.3 s [0.5–4.3]      |
| Time locked, mean $\pm$ 95% CI           | 71.5 $\pm$ 9.5%       | 53.7 $\pm$ 14.4%     |
| Runs with $\geq$ 80% of the run locked   | 55%                   | 50%                  |
| Lock retention after acquisition, median | 100.0%                | 100.0%               |
| Runs with a false lock $>$ 1 s           | 8%                    | 42%                  |
| Mean tracking error while locked, median | 3.56 px               | 4.28 px              |
| Processing time per frame, mean          | 8.1 ms                | 3.7 ms               |

Identification is the single most important element for robustness. It raises the acquisition rate from 62% to 92% and cuts runs with sustained false locks from 42% to 8%. The price is a slower first lock (median 5.2 s vs 2.3 s), because a candidate must accumulate at least 1.6 s of evidence and reach a score of 0.7 before it is confirmed, and more processing per frame, which at 8 ms still leaves ample margin inside the 33 ms frame budget. Once acquired, the lock is typically held for the rest of the run (median retention 100.0%).

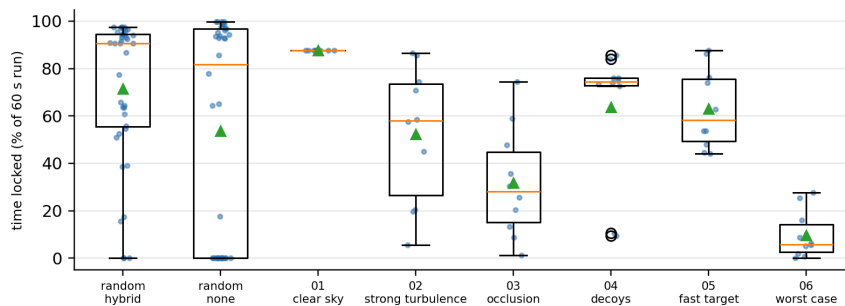
Breaking the results down by scene property (runs/evaluation/summary.md) shows where the remaining difficulty lies. With identification, time locked is 81% for beacons slower than 3.5 °/s and 60% for faster ones, and 76% at turbulence strength below 1.5 against 63% above. More decoys cost little with identification (67% with no decoys, 69% with three or more), whereas the baseline collapses from 70% to 39%.

### 11.3 Fixed scenarios

**Table 11.** Fixed test scenarios with the hybrid identifier, 60 s runs over seeds 1–10 (60 runs in total; time locked as mean  $\pm$  95% CI). The scenario defines the scene; the seed varies noise, turbulence and clouds.

| Scenario             | Runs | Acquired | Acq. time median | Time locked mean | Retention median (visible) | Beacon occluded | Error median |
|----------------------|------|----------|------------------|------------------|----------------------------|-----------------|--------------|
| 01 clear sky         | 10   | 100%     | 7.4 s            | 87.7 $\pm$ 0.0%  | 100.0% (100.0%)            | 0.0%            | 1.44 px      |
| 02 strong turbulence | 10   | 100%     | 11.4 s           | 52.4 $\pm$ 17.9% | 97.1% (97.3%)              | 9.1%            | 4.59 px      |
| 03 occlusion         | 10   | 100%     | 27.5 s           | 31.6 $\pm$ 14.3% | 54.5% (84.0%)              | 48.0%           | 3.86 px      |
| 04 decoys            | 10   | 100%     | 15.1 s           | 63.7 $\pm$ 17.7% | 100.0% (100.0%)            | 7.7%            | 2.70 px      |
| 05 fast target       | 10   | 100%     | 20.3 s           | 63.1 $\pm$ 10.4% | 99.3% (99.3%)              | 0.0%            | 9.63 px      |
| 06 worst case        | 10   | 90%      | 26.8 s           | 9.7 $\pm$ 6.2%   | 29.5% (32.5%)              | 25.8%           | 8.23 px      |

The **clear-sky** baseline acquires in 100% of runs with about 1.4 px mean error; its time locked (88%) is limited mainly by the search and verification time at the start. **Strong turbulence** (2.5 $\times$  turbulence and vibration, 1.5 $\times$  sensor noise) acquires in 100% of runs, but deep fades and large apparent jitter cause lock losses, so time locked varies between seeds. **Occlusion** is dominated by the environment: the beacon is hidden for 48% of the time, yet median retention over the *visible* frames is 84.0%, showing that coasting and predictive re-acquisition work; the low time-locked figure mostly reflects frames in which no camera could see the beacon. The **decoy** scenario acquires the true beacon in 100% of runs, confirming the closed-loop example above across seeds. The **fast-target** scenario (8 °/s random walk with hard turns) acquires in 100% of runs and keeps the lock once gained (median retention 99.3%), but acquisition is slower because the search has to catch a target that moves at two-thirds of the scan speed, and the constant-velocity filter lags the turns ( $\approx$  9.6 px mean error). The **worst-case** scenario stacks every disturbance at once (beacon occluded 26% of the time); the beacon is still acquired in 90% of runs, but locked for only 10% of the time on average, because repeated occlusions, glints and strong turbulence keep sending the tracker back to search. This stress case defines the envelope of the current design and the priorities for further work (Section 13).



**Figure 8.** Distribution of the share of each 60 s run spent locked on the true beacon (box: quartiles, line: median, triangle: mean, dots: individual seeds).

## 11.4 Computational performance

The processing chain (detection, identification with batched CNN inference, tracking and control) takes on average 8.1 ms per  $640 \times 480$  frame with the hybrid identifier on a laptop CPU (3.7 ms without identification), i.e. a throughput of roughly 123 frames/s, comfortably above the 30 frames/s requirement. Rendering the synthetic scene and applying the image-level disturbances is the more expensive half of the loop; it has been optimised with window-limited PSF evaluation, OpenCV routines for full-frame arithmetic, a pre-drawn noise bank and skipped cloud or glare work when nothing is in view. The desktop application runs the loop in real time with a deadline-based scheduler and can also run faster than real time.

## 12. User Interfaces and Tooling

### 12.1 Desktop application

`python app.py` (or the standalone executable) starts a PySide6 application with a dark ‘mission control’ theme and a consistent colour language across all views (cyan = camera, red = beacon, amber = decoys, magenta = estimate, state colours SEARCH red, ACQUIRING orange, TRACKING green, COASTING yellow). The window adapts its layout to the screen size. It contains:

- the **camera feed** with detection boxes, per-candidate  $P(\text{beacon})$  labels (green accepted, red rejected, yellow undecided), the selected candidate, the Kalman estimate with a 0.5 s velocity arrow, the dashed tracking window (ROI), the boresight and the handover circle; a **zoom** (fit, 2 $\times$ , 4 $\times$ ) centred on the tracked beacon shows the few-pixel spot, its blink and the sensor noise with crisp pixels;
- a **situational view** that shows the scene in 3D, as a 2D azimuth / elevation sky map, or both: the 3D view (pyqtgraph OpenGL) shows the pan-tilt terminal whose head turns with the gimbal, the sky dome and the field-of-view pyramid, with depth: a coloured star field far away, and the beacon, decoys and soft cumulus clouds at their simulated distances, each in its true direction; live cues show the disturbances (shimmering air pockets along the line of sight for turbulence, sparkles in the footprint for sensor noise, a visibly shaking camera head for vibration); both views show the camera footprint coloured by tracker state, the history of where the camera has looked (the search spiral becomes visible), the pulsing beacon (label ‘B’) and its trail, decoys, clouds, stars and the estimate; with 3D camera presets, auto-rotation and an ‘i’ button that opens the legend;
- **live plots** of pointing error (log scale, locked periods shaded, handover line) and of the followed candidate's beacon score with the accept and reject thresholds;
- **controls** for scenario, **sky area** (standard  $\text{az} \pm 30^\circ \times \text{el} \pm 20^\circ$  or wide  $\text{az} \pm 60^\circ \times \text{el} \pm 30^\circ$ ), identification method, **decoy range** (from A to B, 2–8), run length, simulation speed (0.5 $\times$  to maximum), sliders for turbulence, vibration and sensor noise with **Apply & continue** (the new levels take effect from the current moment of the run) or **Apply & restart** (the same scene from the beginning), display options and report export, and an ‘i’ button that explains each disturbance with pictures generated at the current levels; and **performance cards** with a description of the current scene.

The application can be operated entirely from the keyboard, without a mouse: Space start/pause, R restart, N new random scene, C / Shift+R apply disturbances and continue / restart, + / - / 0 camera zoom, M switch between 3D, 2D and both views, A / D and W / S rotate and tilt the 3D view, Q / E zoom it, V next preset, I legend, F2 disturbances explained, H or F1 list of shortcuts, Tab and the arrow keys for all other controls.

The simulation runs in the GUI thread paced by a 5 ms timer that advances as many frames as real time  $\times$  speed requires, capped by a 28 ms budget per tick so the interface stays responsive. At the end of a run a performance

report is saved automatically as `report.html` (interactive 3D views) and `report.pdf`, together with `frames.csv` and the summary files.

## 12.2 Standalone executable

`tools/build_exe.py` packages the application with PyInstaller from a minimal virtual environment (no PyTorch: the CNN runs through OpenCV DNN), in two variants: a single file `FSOC_Simulator.exe` (about 100 MB) and a folder version that starts faster. Both open only the application window (no console) and write their reports to a `runs` folder next to the executable. `FSOC_Simulator.exe --selftest 20` runs a short headless simulation and writes a pass/fail result, so a build can be checked without the GUI.

## 12.3 Command line and tools

```
python main.py                                # OpenCV viewer, random scene
python main.py --headless --duration 120 --seed 7 # batch run, report only
python tools/generate_dataset.py                # labelled clips for the CNN
python tools/train_classifier.py                 # train, evaluate, export ONNX
python tools/evaluate.py --suite random --seeds 1-40 --methods hybrid none
```

The evaluation tool appends each run to `results.csv` immediately, so an interrupted evaluation resumes where it stopped, and it produces summary tables with Wilson confidence intervals for rates and normal-approximation intervals for means, breakdowns by scene property and box plots.

## 13. Limitations and Future Work

The following limitations were identified while reviewing the code and results, in order of expected impact:

- **Severe combined conditions.** In the worst-case scenario the tracker is locked for only 10% of the time: after each occlusion it must search and re-verify the beacon from scratch. Keeping the identity of a confirmed beacon across short losses (re-acquiring a candidate at the predicted position without the full 1.6 s verification) is the most promising improvement.
- **Motion model.** The constant-velocity Kalman filter lags hard manoeuvres; the fast-target results show it. An interacting-multiple-model (IMM) filter (constant velocity plus coordinated turn or constant acceleration) is the standard remedy and fits into the existing tracker interface.
- **Acquisition latency.** Identification needs at least 1.6 s of evidence before confirming. Shorter windows, a sequential probability ratio test with explicit error rates, or a higher-rate beacon code would shorten this.
- **Geometry.** The camera uses a linear small-angle projection and the world is a flat az/el patch. This is adequate for an 8° field of view and the standard  $\pm 30^\circ \times \pm 20^\circ$  sky; on the wide  $\pm 60^\circ \times \pm 30^\circ$  sky the flat-patch approximation distorts directions near the edges, and a wide-FOV camera or near-zenith pointing would need a proper spherical (gnomonic) projection and gimbal kinematics.
- **Physical fidelity.** Turbulence is parameterised directly in pixels and log-intensity rather than derived from  $C_n^2$  and path length; the sensor is 8-bit with a simple noise model. Linking these parameters to a link budget (range, wavelength, aperture) would make the simulator quantitatively predictive.
- **Training data.** Glints almost never survive long enough to form a 24-frame sample (4 in dataset v2), so they are rejected by the tracker's verification rather than by the classifier.
- **Scope of the loop.** Only the coarse stage is simulated; the fine-pointing handover is represented by a radius criterion. Closing the loop with a fast-steering-mirror model, and with the beacon on a real partner terminal that tracks back, would complete the PAT picture.
- **Evaluation size.** Results are based on 40 random scenes per method and 10 seeds per scenario. The confidence intervals in Tables 10 and 11 are reported with every figure, and the between-seed spread (Figure 8) is as informative as the means.



## 14. Conclusion

The project delivers a complete, reproducible software test bench for the coarse stage of FSOC pointing, acquisition and tracking. The simulated camera and environment reproduce the effects that make coarse tracking hard, each switchable and scalable, and the closed loop implements the full PAT sequence from an open-loop pointing error through search, verification, tracking, coasting and re-acquisition. The key technical contribution is identity-aware tracking: candidates are followed over time, scored by a blink-frequency test and a compact spatio-temporal CNN, and the evidence is accumulated before the tracker accepts or rejects them. In 40 random scenes this raises the acquisition rate from 62% to 92% and cuts runs with sustained false locks from 42% to 8%, at 8 ms per frame, well within real-time limits. The system hands the beacon over inside the 2.2 mrad capture zone of a fine-pointing stage. Every run is logged against ground truth with strict metrics, and the evaluation tooling reports results over many seeds with confidence intervals, providing a sound basis for further tuning and for comparing new algorithms.

## Appendix A. Key default parameters

*Table 12. Default configuration (config/default.yaml).*

| Group          | Parameter                                                           | Value                                                                  |
|----------------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| Simulation     | Frame rate / default duration                                       | 30 fps / 60 s                                                          |
| Random scenes  | Beacon speed, decoys, turbulence, vibration, clouds, pointing error | 0.8–5 °/s, 2–5, 0.5–2.0, 0.3–2.0, $p = 0.5$ , 3–10°                    |
| Camera         | Resolution, FOV, pixel scale                                        | 640 × 480, 8° × 6°, 0.0125° = 0.218 mrad per px                        |
| Gimbal         | Max rate / acceleration                                             | 25 °/s / 80 °/s <sup>2</sup>                                           |
| Beacon         | Blink code                                                          | 4 Hz, duty 0.7, off level 0.3                                          |
| Detector       | Global / ROI threshold                                              | $\max(b + 8\sigma, 80) / \max(b + 5\sigma, 45)$ , ROI $\pm 40$ px      |
| Identification | Clip, evidence memory, accept / reject                              | 24 frames, 90 frames, 0.7 / 0.2                                        |
| Identification | Verify limit, drop rule, blacklist                                  | 90 frames, < 0.35 for 30 frames, 4 s                                   |
| Tracker        | Acquire hits, hold, coast                                           | 5 hits, 6 frames, 60 frames or $\sigma > 1^\circ$                      |
| Tracker        | q, R, gates                                                         | 3 °/s <sup>2</sup> , (0.02°) <sup>2</sup> , $\chi^2 < 11.6$ or < 0.15° |
| Controller     | Kp, Ki, Kd, feed-forward, anti-windup                               | 5, 0.5, 0.05, 1.0, $\pm 2^\circ$ ·s                                    |
| Search         | Spacing, speed, radius, passes, drift $\tau$                        | 4.8°, 12 °/s, 15°, 4 then position report, then raster, 3 s            |
| Metrics        | Lock tolerance, handover radius                                     | 8 px, 10 px                                                            |